

DISTRIBUTED DATA PROCESSING

Introduction to PySpark

PySpark in Action: Building scalable, high-performance data pipelines
with Apache Spark

What is Apache Spark & PySpark?



Unified Engine

Distributed cluster computing engine for massive analytical tasks.



In-Memory Speed

RAM caching eliminates disk bottlenecks for fast execution.



Python API

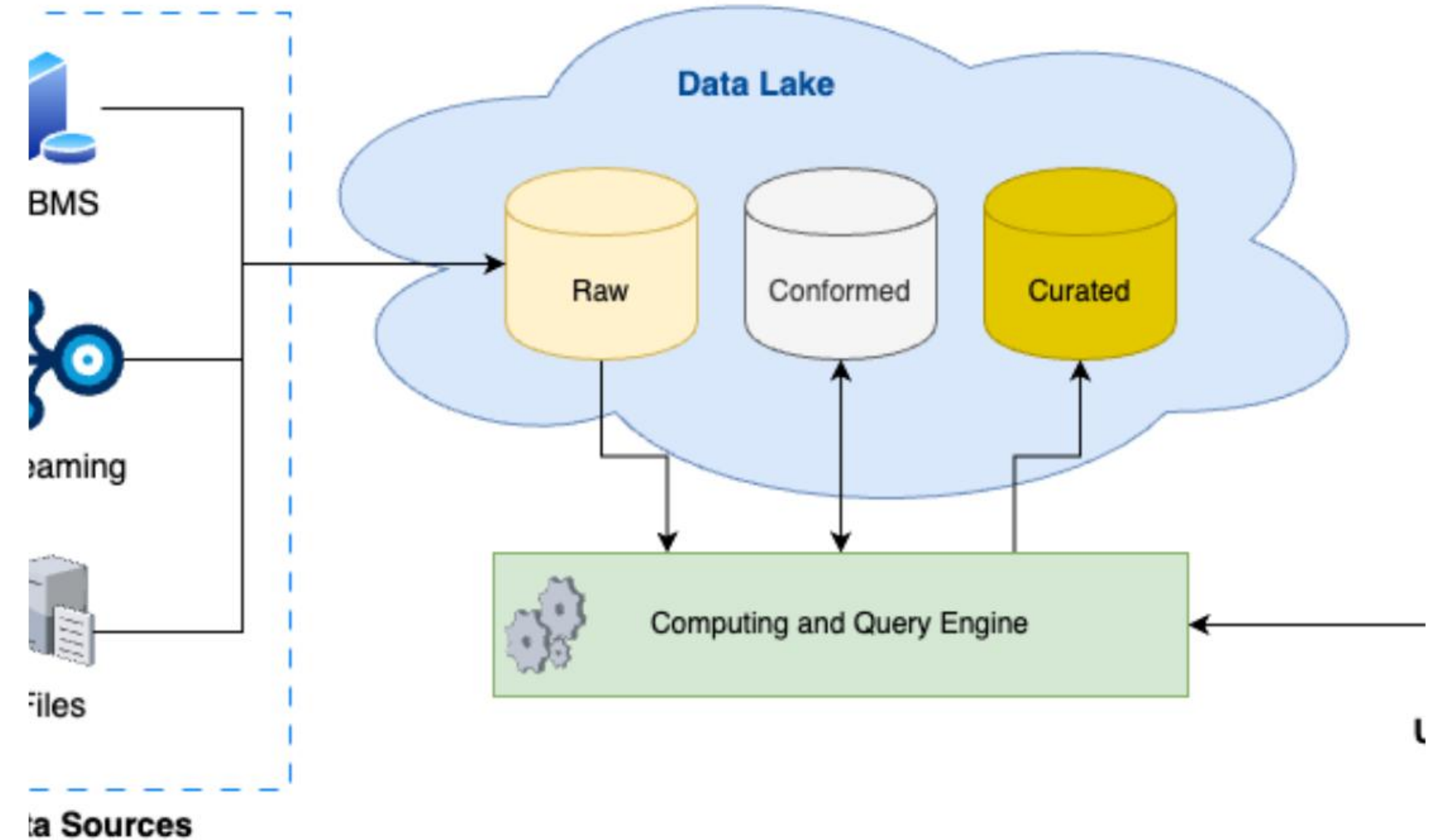
PySpark exposes Spark Scala performance to Python developers.

Spark Cluster Architecture

Driver Node: Orchestrates execution, creates DAG graphs, schedules worker tasks.

Cluster Manager: Allocates resources across YARN, Kubernetes, or Standalone clusters.

Executor Nodes: Execute task partitions in parallel and cache in-memory data.



SparkSession: Unified Entry Point

Python Session Initialization

```
from pyspark.sql import SparkSession

# Build or retrieve existing session
spark = (
    SparkSession.builder
    .appName("PySparkInAction")
    .config("spark.executor.memory", "4g")
    .getOrCreate()
)

# Access low-level context
sc = spark.sparkContext
```

Unified Access

Combines SQLContext, HiveContext, and SparkContext.

Declarative Config

Sets runtime memory, cores, and package extensions.

RDDs vs. Modern DataFrames

Resilient Distributed Datasets (RDD)

Low-Level API: Functional lambda transformations.

Unstructured: No built-in relational schema awareness.

Manual Tuning: Lacks automatic execution plan optimization.

PySpark DataFrames

High-Level API: Tabular structure with named typed columns.

Catalyst Engine: Automatic physical query optimization.

Tungsten Memory: Off-heap memory storage and binary encoding.

Multi-Format Data Ingestion

CSV / Text

Schema inference with header parsing options.

JSON Lines

Handles semi-structured nested documents seamlessly.

Apache Parquet

Columnar binary format with snippet compression.

Delta Lake

ACID storage layer with time-travel versioning.

Essential DataFrame Operations

```
from pyspark.sql.functions import col, when

# Select, filter, and derive new columns
processed_df = (
    df.select("user_id", "status", "amount")
      .filter(col("status") == "COMPLETED")
      .withColumn("tier",
                  when(col("amount") > 500, "PREMIUM")
                  .otherwise("STANDARD"))
)
```

Column Projections

Pushes column filtering down to source readers.

Conditional Logic

Evaluates fast vectorized IF-ELSE branching.

Transformations vs. Actions

Transformations (Lazy)

Examples: `filter()`, `select()`, `groupBy()`

Behavior: Returns a new DataFrame without computing.

DAG Building: Records computational graph instructions.

Actions (Eager)

Examples: `count()`, `collect()`, `write()`

Behavior: Triggers execution across the Spark cluster.

Materialization: Produces driver outputs or writes storage.

Aggregations & Grouping

```
from pyspark.sql import functions as F

# Grouping and multiple column aggregations
summary_df = (
    df.groupBy("region", "category")
        .agg(
            F.count("order_id").alias("total_orders"),
            F.sum("amount").alias("total_revenue"),
            F.avg("amount").alias("avg_spend")
        )
)
```

Map-Side Combine

Reduces local partition records before network transfer.

Built-in Expressions

Optimized Scala expression execution engine.

Joins & Shuffle Mechanics



Shuffle Hash Join

Exchanges data over network partitions for large table joins.



Broadcast Join

Replicates small dimension lookup tables to every worker executor.



Sort-Merge Join

Sorts keys before joining massive skewed datasets cleanly.

Optimization Pillars

Partitioning

Aligns tasks with data blocks using `repartition()`.

Caching & Persist

Saves intermediate dataframes in memory using `cache()`.

AQE Engine

Adaptive Query Execution optimizes runtime shuffle plans.

PySpark SQL Integration

```
# Register DataFrame as temporary SQL view
df.createOrReplaceTempView("transactions")

# Run pure ANSI SQL queries directly
result_df = spark.sql("""
    SELECT region, COUNT(*) as total_tx
    FROM transactions
    WHERE status = 'SUCCESS'
    GROUP BY region
    HAVING total_tx > 100
""")
```

ANSI Interoperability

Blends standard SQL statements with Python code pipelines.

Shared Optimization

Translates SQL queries directly into Catalyst RDD plans.

PySpark MLlib Pipelines



Feature Transformers

Combines input feature columns using [VectorAssembler](#).



Distributed Estimators

Trains parallel models using Random Forest or Logistic Regression.



Pipeline Execution

Chains transformation stages into reproducible workflows.

Structured Streaming Architecture

STAGE 1

Stream Source

Reads live data streams from Kafka or Event Hubs.

STAGE 2

Micro-Batching

Processes continuous input streams in discrete batches.

STAGE 3

State Management

Tracks event-time aggregations with watermarking.

STAGE 4

Sink Output

Writes results out to Delta, Parquet, or databases.

PySpark in Action Architecture



1. Ingestion

Reads S3/HDFS storage with explicit schema definition.



2. Wrangling

Applies column casting, null handling, and string cleansing.



3. Aggregation

Executes window functions and grouped metrics calculation.



4. Delta Storage

Saves partitioned Delta tables for fast downstream analytics.

SUMMARY & NEXT STEPS

Mastering PySpark in Action

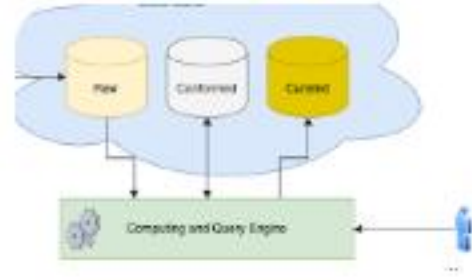
1. Leverage DataFrames: Prefer DataFrame/SQL APIs over low-level RDD transformations.

2. Optimize Shuffles: Use broadcast joins and tune partition sizes to prevent skew.

3. Unify Workloads: Run batch, streaming, and MLlib modeling on one SparkSession.

Ready to scale your Python data pipelines?

Image Sources



https://miro.medium.com/v2/resize:fit:666/1*vh1hFhWU0w0CzCiA64Xy3A.png

Source: medium.com